

ОБЪЕКТНО- ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ



РЕАЛИИ НАШЕГО ДНЯ

- ООП везде. Из **10** самых популярных языков программирования **7** поддерживают ООП на уровне синтаксиса.

<http://www.tiobe.com/index.php/content/paperinfo/tpci/index.html>

- Программы на C++ работают быстро.

О КУРСЕ

Две основных темы:

- C++.
- Принципы ООП.

Орг. часть:

- «Как на ОПК».
- *I часть:* проект, теория, семинары => диф. зачет + экзамен.
- *II часть:* проект, теория, семинары, *I часть* => экзамен.

О КУРСЕ

Основные Правила семинаров



<http://oop.afti.ru/materials/257>

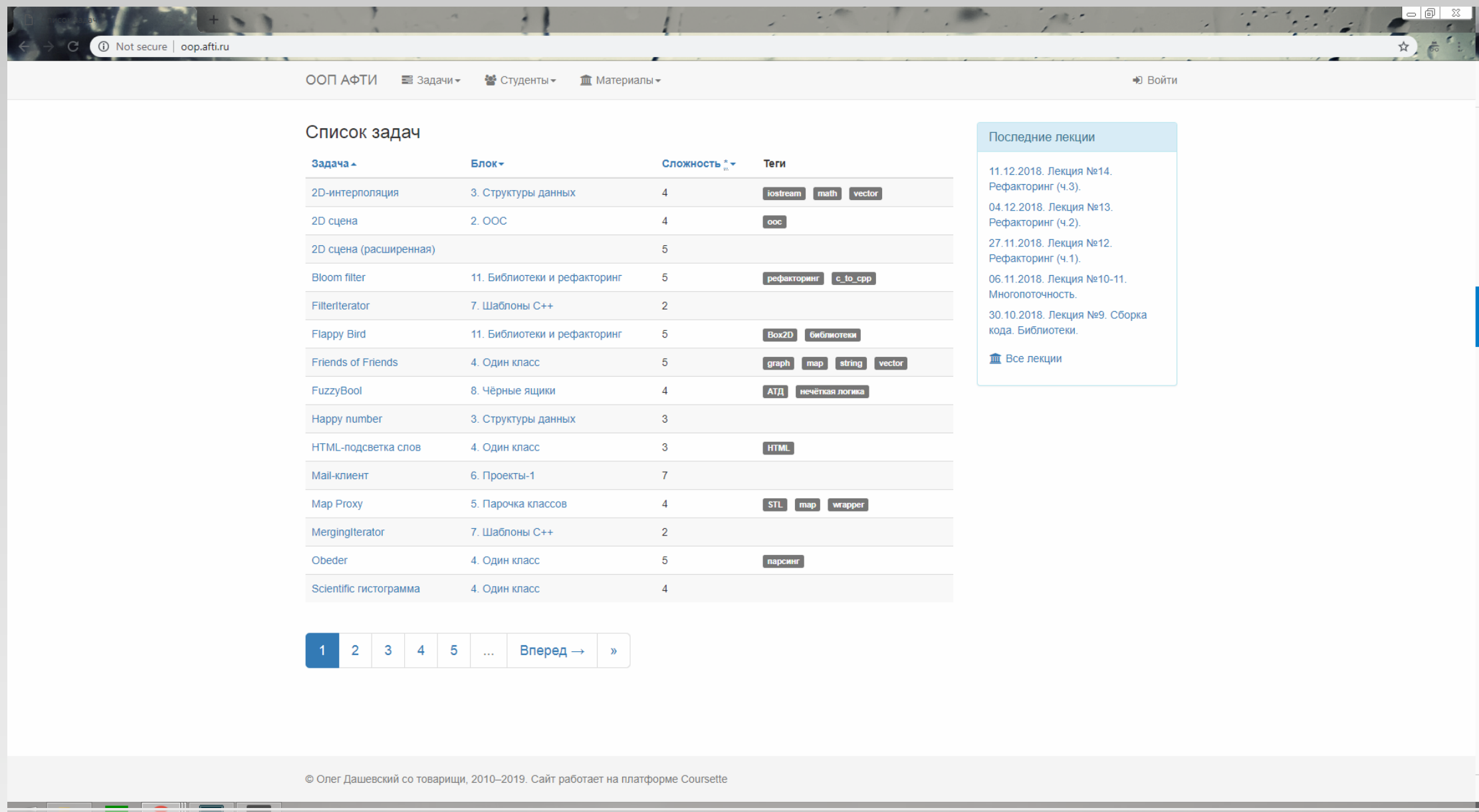
О КУРСЕ

Семинаристы:

Барамия Денис Александрович
d.baramiia@g.nsu.ru

Шишкин Дмитрий Юрьевич
shish@sl.iae.nsk.su

Гузенкова Анна Андреевна
a.guzenkova@g.nsu.ru



<http://oop.afti.ru/>

ЛАДНО, КРУТО!

А ЧТО ТАКОЕ ООП?

A person with long brown hair is wearing a white t-shirt. The t-shirt has a black text print on the chest. The text is a quote about object-oriented programming, attributed to Edsger Dijkstra. The person is also wearing blue jeans.

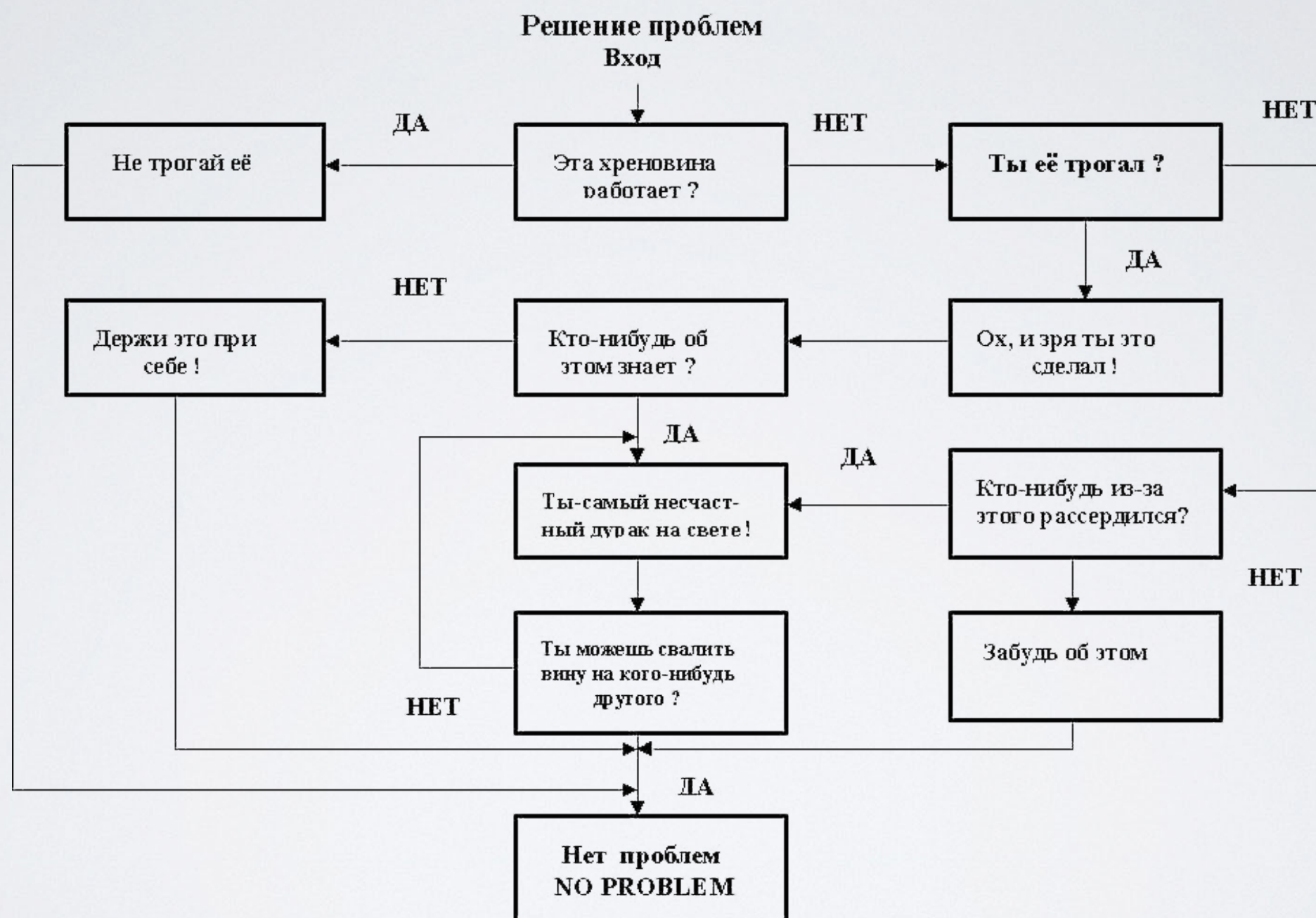
"Object-oriented programming
is an exceptionally bad idea
which could only have
originated in California."
-- Edsger Dijkstra

ПАРАДИГМЫ

Известные нам парадигмы:

- императивная (алгоритмическая);
- функциональная;
- логическая.

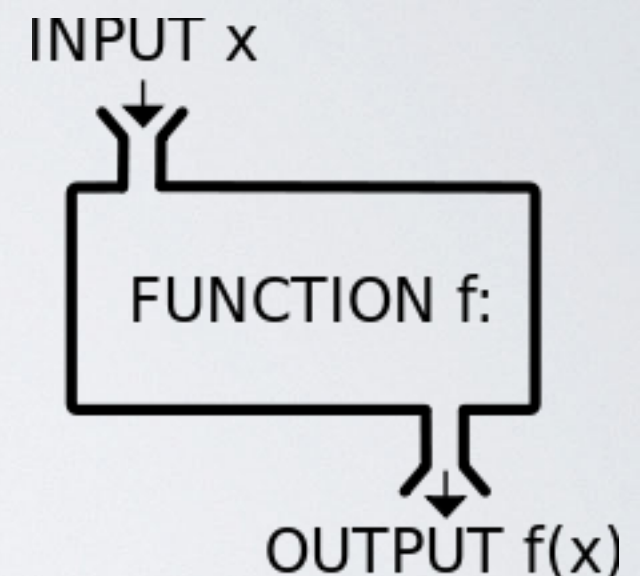
ИМПЕРАТИВНАЯ ПАРАДИГМА



последовательность действий

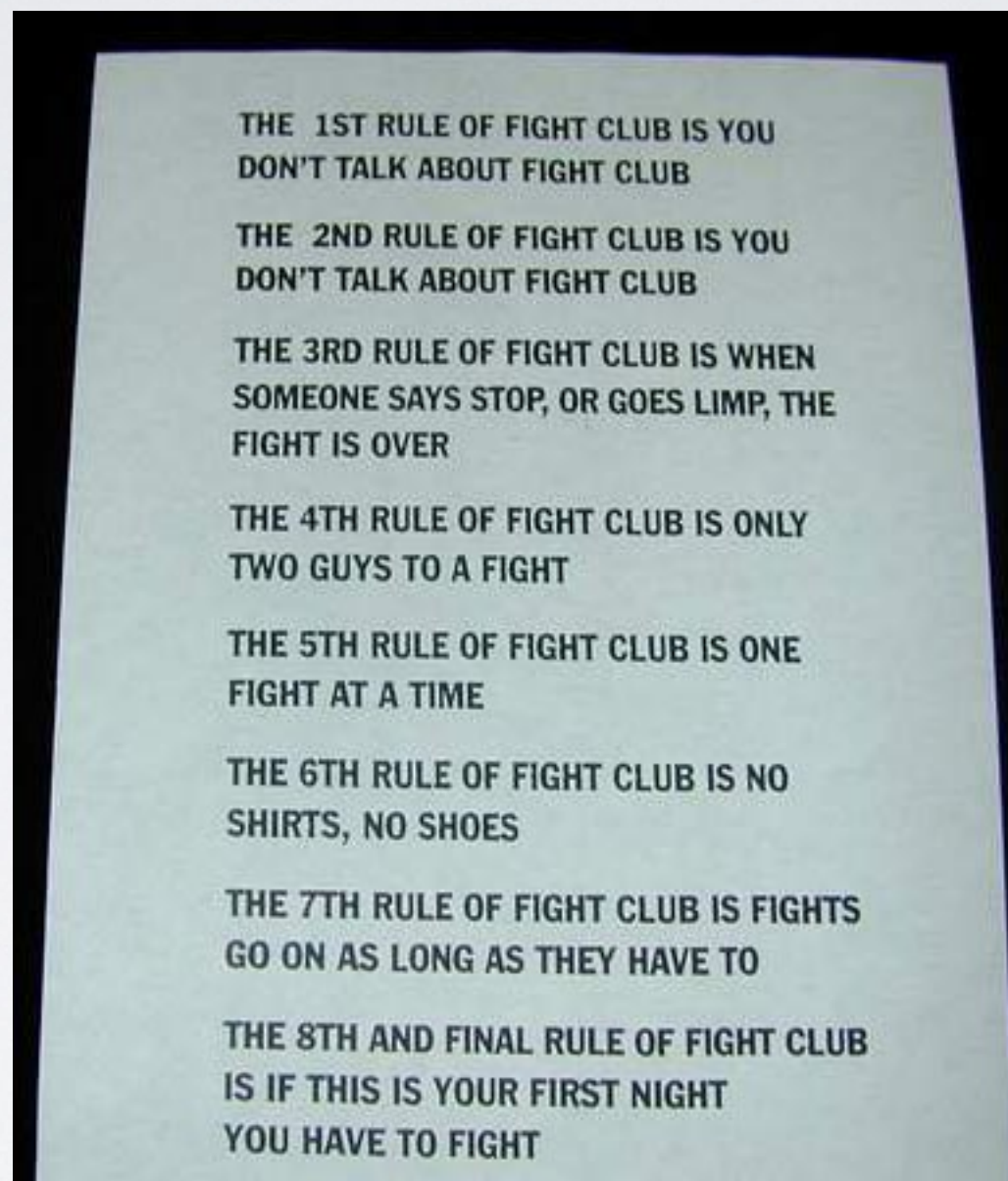
ФУНКЦИОНАЛЬНАЯ ПАРАДИГМА

- $\text{result} = f1(a, b) + f2(b, c)$
- Данные являются неизменяемыми.
- Последовательность действий не важна.
- Текущее состояние не важно.



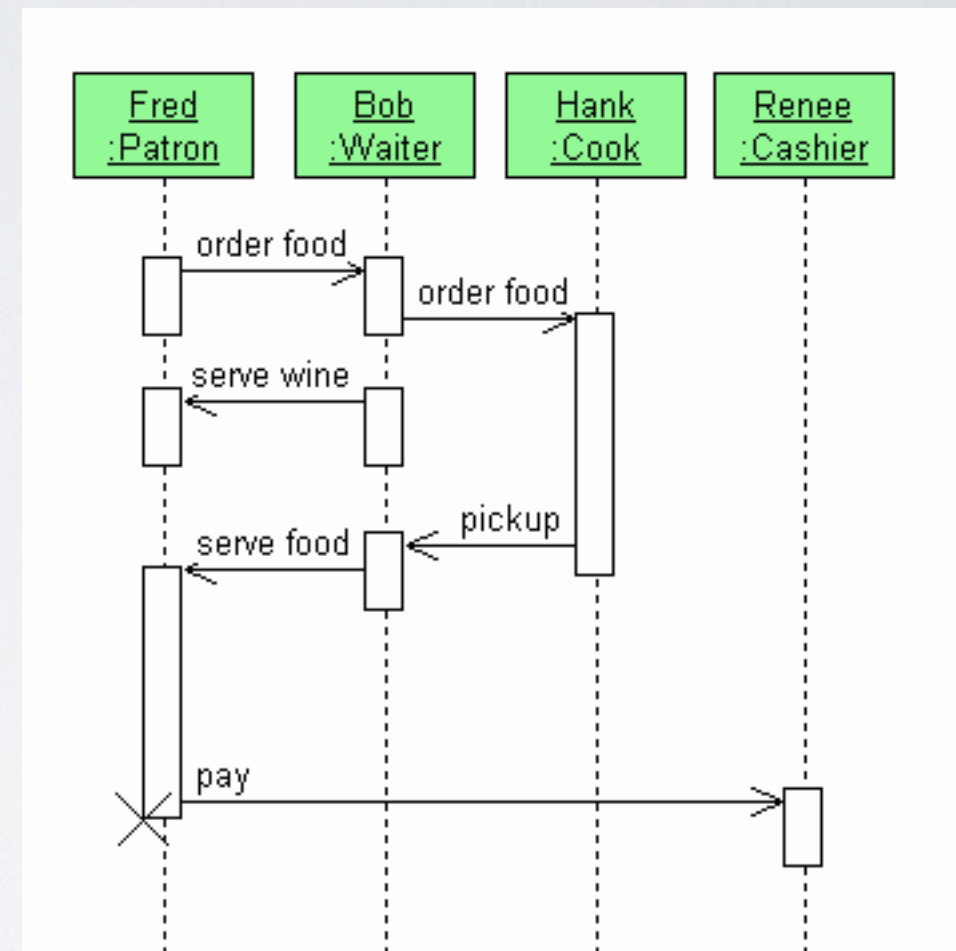
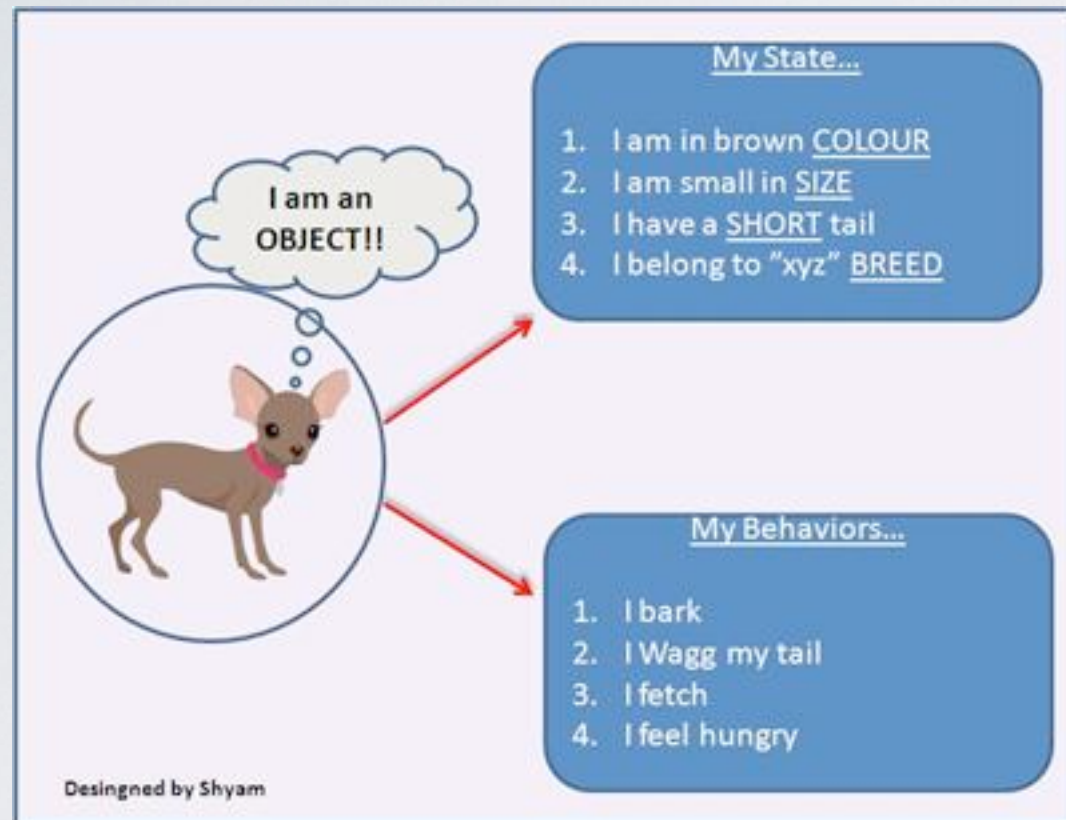
данные и преобразования над ними

ЛОГИЧЕСКАЯ ПАРАДИГМА



предикаты, правила

О.О. ПАРАДИГМА



объекты, их «поведение» и взаимодействие

ОБЪЕКТЫ

- Программы так или иначе моделируют мир.
- В мире есть «объекты», т. е. некие отдельные сущности.
- Пусть в программах тоже будут объекты!
- Для нас важны **свойства** и **поведение** объекта исходя из решаемой задачи !!

ВЗАИМОДЕЙСТВИЕ ОБЪЕКТОВ

- Важен момент **рождения** объекта и время его **жизни**.
- Принцип **иерархии**: есть более глобальные объекты, которые живут дольше и порождают менее глобальные объекты.
- Принцип **черного** ящика: взаимодействие с любым объектом через интерфейс.

КАКОЙ ЯЗЫК
САМЫЙ ООП-ИСТЫЙ?



ЧЕТЫРЕ КИТА ООП

Абстракция

Полиморфизм

Наследование

Инкапсуляция

ИНКАПСУЛЯЦИЯ

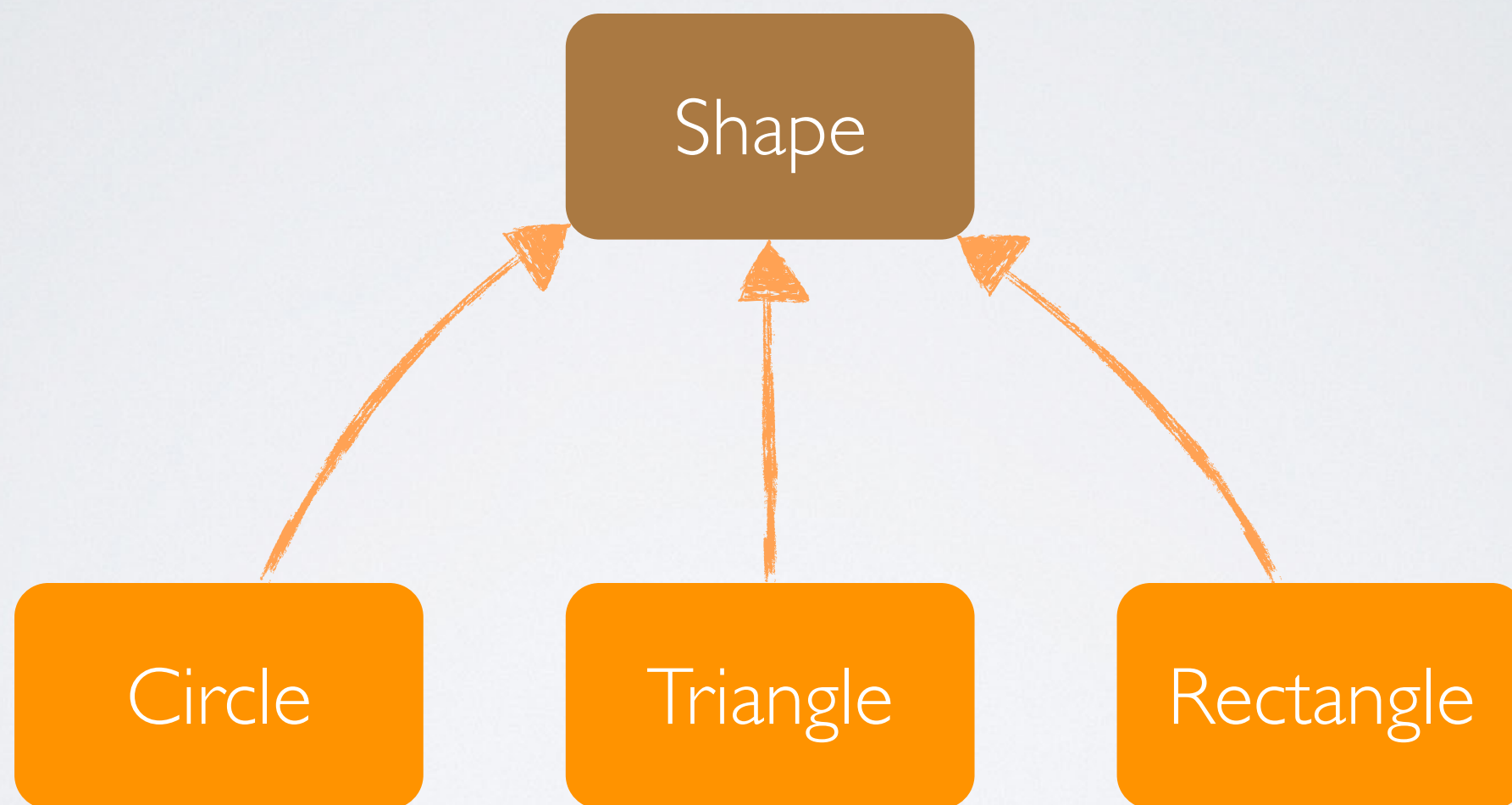
- Данные сокрыты.
- Детали реализации сокрыты.
- Наружу торчит только интерфейс.



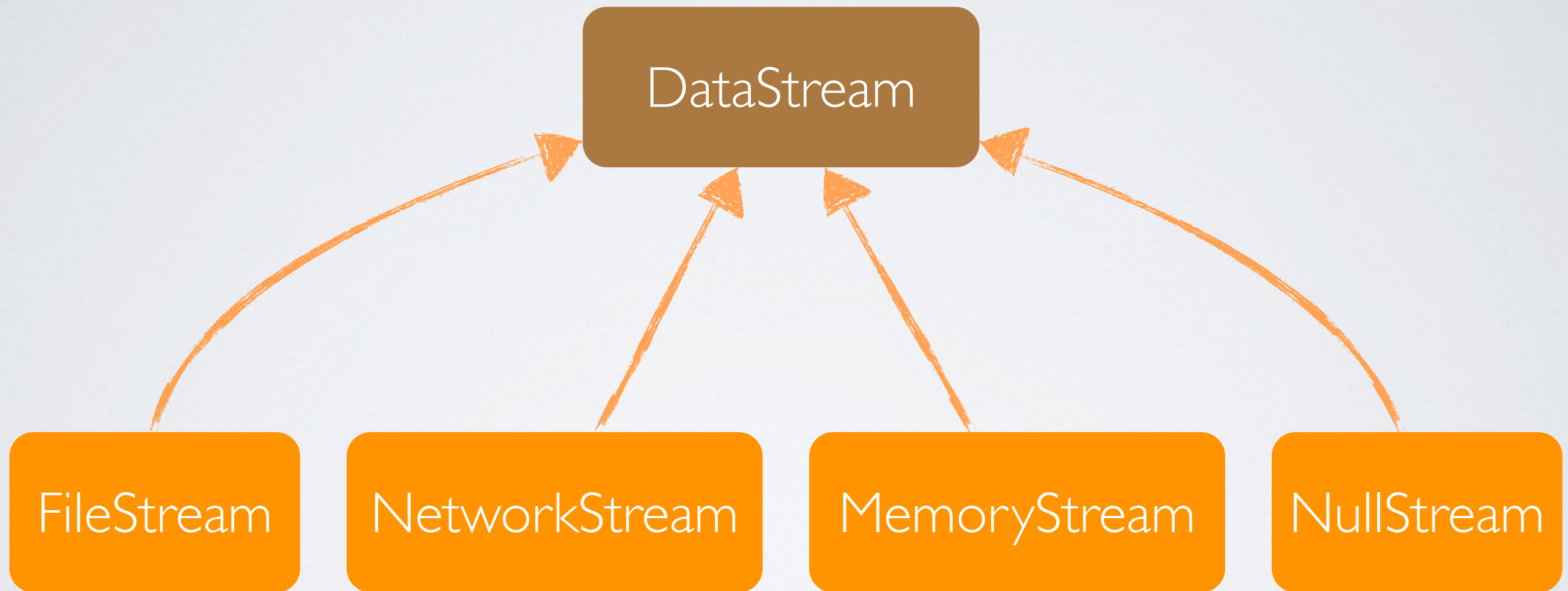
НАСЛЕДОВАНИЕ

- «**Этот** объект ведет себя так же, как **тот**, только немного по-другому»
- Поведение класса **A** наследуется от класса **B** (с изменениями).
- **B** — базовый класс (суперкласс).
- **A** — производный класс (подкласс).

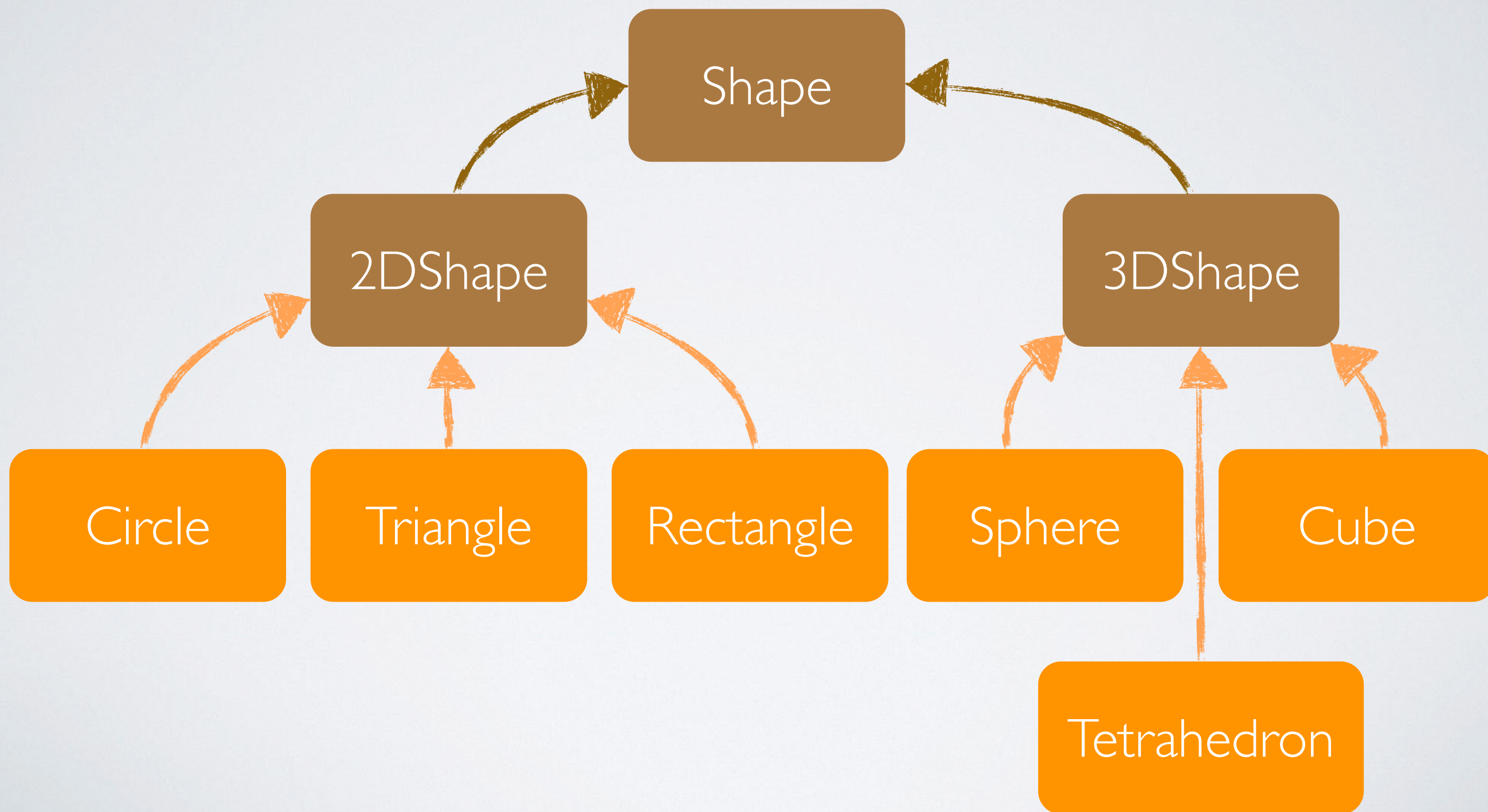
ГЕОМЕТРИЧЕСКИЕ ФИГУРЫ



ПЕРЕДАЧА ДАННЫХ



ИЕРАРХИЯ НАСЛЕДОВАНИЯ



ПРИНЦИПЫ НАСЛЕДОВАНИЯ

- Принцип «**A** является **B**» (*is-a*). Круг (**Circle**) является плоской фигурой (**2DShape**), которая является просто фигурой (**Shape**). Обратное неверно!
- Нельзя путать с «**A** содержит **B**» (*has-a*). Это не наследование, а композиция!

НЕКОТОРЫЕ ПРОБЛЕМЫ

- Square **is a** Rectangle? (Circle **is a** Ellipse?)
 - Будет ли удачным ходом сделать класс **Square** наследником **Rectangle**?
 - Ответ: **нет**, если объект может изменяться. Методы **stretchWidth** и **stretchHeight** не годятся для **Square**.
 - Ответ: **да**, если объект не будет изменяться после создания.
- 
- ValueObject

ПОЛИМОРФИЗМ

Один интерфейс, много реализаций.

Полиморфизм

```
int abs(int x){}  
  
double abs(double x){}  
  
float abs(float x){}  
  
long abs(long x){}  
  
long long abs(long long x){}
```

Без полиморфизма

```
int abs(int x){}  
  
double fabs(double x){}  
  
float fabsf(float x){}  
  
long labs(long x){}  
  
long long llabs(long long x){}
```


АБСТРАКЦИЯ



Абстракция кота

Значимое поведение

- Мяукнуть
- Поесть
- Потребовать погладить
- Погулять

Незначимое поведение

- Рвать обои
- Испортить тапки



Вспомним немного С

СТРУКТУРЫ

```
// point.h
struct Point {
    int x, y;
};
```

Это скомпилируется?

```
// main.c
int main() {
    Point point = {1, 1};
    printf("Point(%d, %d);\n",
        point.x, point.y);
    return 0;
};
```


СТРУКТУРЫ

```
// point.h
struct Point {
    int x, y;
};

// main.c
int main() {
    struct Point point = {1, 1};
    printf("Point(%d, %d);\n",
        point.x, point.y);
    return 0;
};
```

не очень удобно



СТРУКТУРЫ

```
// point.h
struct Point {
    int x, y;
};

// main.c
int main() {
    struct Point point = {1, 1};
    printf("Point(%d, %d);\n",
        point.x, point.y);
    return 0;
};
```

не очень удобно

```
// point.h
typedef struct {
    int x, y;
} Point;

// main.c
int main() {
    Point point = {1, 1};
    printf("Point(%d, %d);\n",
        point.x, point.y);
    return 0;
};
```

лучше

EXTERN

```
// file.h
```

```
extern double PI; // <- declaration
```

```
// file.c
```

```
#include "file.h"
```

```
double PI = 3.14159265359; // <- definition
```

```
// main.c
```

```
#include "file.h"
```

```
int main() {  
    printf("PI = %.11f;\n", PI);  
    return 0;  
};
```

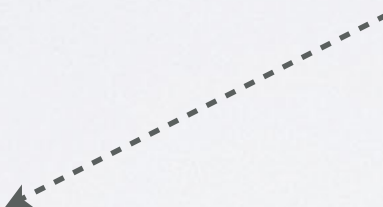

STATIC

```
// file.h  
int counter();
```

```
// file.c  
#include "file.h"
```

```
int counter() {  
    static int count = 0;  
    return count++;  
}
```

Область видимости
ограничена блоком, в котором
определен static



```
// main.c  
#include "file.h"
```

```
int main() {  
    printf("%d\n", counter());  
    printf("%d\n", counter());  
    return 0;  
};
```

INCLUDE GUARD

```
// point.h
#ifndef POINT_H
#define POINT_H

struct Point {
    int x, y;
};

#endif//POINT_H
```

```
// circle.h
#include "point.h"

// main.c
#include "point.h"
#include "circle.h"
```

PRAGMA ONCE

```
// point.h  
#pragma once
```

```
struct Point {  
    int x, y;  
};
```

```
// circle.h  
#include "point.h"
```

```
// main.c  
#include "point.h"  
#include "circle.h"
```


УКАЗАТЕЛИ НА ФУНКЦИИ

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int (*func_ptr)();
```

```
    func_ptr = printf;
```

```
    (*func_ptr)("Printf is here!\n");
```

```
}
```

/ Объявление указателя */*

/ Присваивание */*

/ Вызов функции */*

```
void qsort(void *base, size_t nitems,  
           size_t item_size,  
           int (*compar)(void *p1, void *p2));
```

```
void *bsearch(void *key, void *base,  
              size_t nitems, size_t item_size,  
              int (*compar)(void *p1, void *p2));
```

- **base** — указатель на начало массива.
- **nitems** — количество сортируемых элементов.
- **item_size** — размер одного элемента в байтах.
- **compar** — функция сравнения. Возвращает **-1**, **0** или **1**. **p1** и **p2** — указатели на сравниваемые элементы.

```
#include <stdio.h>
#include <stdlib.h>
```

```
char *names[] = {
    "Vasya",
    "Petya",
    "Semyon Semenych"
};
```

```
int name_compare(void *p1, void *p2) {
    char *n1 = *(char **)p1;
    char *n2 = *(char **)p2;

    return strcmp(n1, n2);
}
```

```
int main()
{
    int i;
    qsort(names,                                // void *base
           sizeof (names) / sizeof (char *),    // size_t n_items
           sizeof (char *),                     // size_t item_size
           name_compare);                       // int (*compar)(void *p1, void *p2)

    for (i = 0; i < 3; ++i)
        puts(names[i]);
}
```


ВИДЫ ПАМЯТИ В С



ВИДЫ ПАМЯТИ В С

- глобальная (статическая); ← - - - - - static
- автоматическая (стековая); ← - - - - - stack
- динамическая (**malloc** / **free**). ← - - - - - heap

ПАМЯТЬ В С-ПРОГРАММЕ

Скомпилированный код

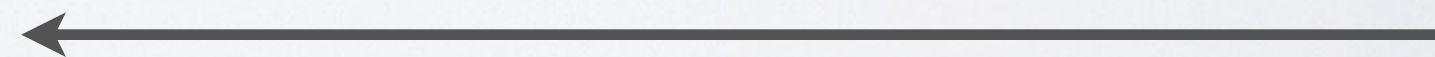
Только чтение
Размер фиксирован

Глобальная память (вкл. static)

Чтение / запись
Размер фиксирован
Время жизни переменных: максимально

Свободный стек

Занятый стек



Чтение / запись
Размер динамический, обычно ограничен сверху
(задается в свойствах линкера)
Время жизни переменных = время жизни фрейма

СТЕКОВЫЕ ФРЕЙМЫ

```
void MyFunction3(int x, int y, int z)
{
    int a, int b, int c;
    a = 10;
    ...
    return;
}
```

```
_MyFunction3:
    push ebp
    mov ebp, esp
    sub esp, 12 ; sizeof(a) + sizeof(b) + sizeof(c)
    ; x = [ebp + 8], y = [ebp + 12], z = [ebp + 16]
    ; a = [ebp - 4] = [esp + 8], b = [ebp - 8] = [esp + 4],
    ; c = [ebp - 12] = [esp]
    mov [ebp - 4], 10
    ...
    mov esp, ebp
    pop ebp
    ret 12 ; sizeof(x) + sizeof(y) + sizeof(z)
```

MyFunction3(10, 5, 2); →

```
push 2
push 5
push 10
call _MyFunction3
```

стек растёт в направлении
меньших адресов памяти

ebp+16	Аргумент z
ebp+12	Аргумент y
ebp+8	Аргумент x
ebp+4	Адрес возврата
ebp	Старый ebp
ebp-4	Переменная a
ebp-8	Переменная b
ebp-12	Переменная c
esp + 4	Свободная ячейка
....	Свободная ячейка



ПЕРЕМЕННОЕ ЧИСЛО АРГУМЕНТОВ

```
#include <stdarg.h>

double average(int n, ...)
{
    va_list myList;
    va_start(myList, n);

    // ...

    va_end(myList);
}

average(5, 1.0, 2.0, 3.0, 4.0, 5.0);
```

ebp+44	Аргумент 5.0
ebp+36	Аргумент 4.0
ebp+28	Аргумент 3.0
ebp+20	Аргумент 2.0
ebp+12	Аргумент 1.0
ebp+8	Аргумент n
ebp+4	Адрес возврата
ebp	Старый ebp
ebp-4	Переменная myList
....	

```
#include <stdarg.h>
```

```
double average(int n, ...)
```

```
{
```

```
    va_list myList;
```

```
    va_start(myList, n);
```

```
// Инициализация
```

```
    int numbersAdded = 0;
```

```
    double sum = 0;
```

```
    while (numbersAdded < n) {
```

```
        double number = va_arg(myList, double); // Очередное число
```

```
        sum += number;
```

```
        numbersAdded++;
```

```
    }
```

```
    va_end(myList);
```

```
    return sum / numbersAdded;
```

```
// Вычисляем среднее
```

```
}
```


УЧИМСЯ ПРАВИЛЬНО ОФОРМЛЯТЬ КОД

<https://tproger.ru/translations/learn-basic-c-coding-rules-from-open-source-projects/>



Программирование - это размышление, а не печатание! (Кейси Патон)



Программирование - это размышление, а не печатание! (Кейси Патон)

Пишите код так, как будто сопровождать его будет склонный к насилию психопат, который знает, где вы живёте! (Джон Вудс)

КОНЕЦ ПЕРВОЙ ЛЕКЦИИ

